

高级语言程序设计大作业实验报告

目录

一. 作业题目 二. 开发软件 三. 课题要求 四. 主要流程 五. 单元测试 六. 收获与体会

- 整体流程
- 核心模块设计
- 核心机制与算法

一. 作业题目

高考模拟器 - 基于 EasyX 图形库的轻量休闲小游戏

二. 开发软件

- 集成开发环境: Visual Studio 2022 Community
- 图形库: EasyX Graphics Library 2024.1

三. 课题要求

3.1 课程通用要求

- 使用 C++ 语言完成一个图形化小程序
- 图形化平台不限, 本项目选用 EasyX 图形库
- 程序主题自选, 注重创新与创意
- 代码结构清晰, 具有良好的可读性和可维护性

3.2 本项目具体要求

- 实现高考模拟的核心玩法: 30 天学习周期, 平衡学习与休息
- 实现图形化界面和鼠标交互操作, 通过点击场景内物品 (课本、黑板、手机) 触发行动
- 包含分科目成长体系 (语、数、英、理综), 不同玩法对应不同科目提升
- 实现答题系统: 日常刷题、月考、高考三种答题模式, 限时作答
- 实现随机事件系统、疲劳值机制、志愿填报系统和多结局系统
- 保证程序运行稳定, 无内存泄漏和崩溃问题
- 游戏流程完整, 从开始到结局体验流畅

四. 主要流程

1. 整体流程

本项目采用状态驱动的游戏架构, 整体流程如下:

plaintext

初始化游戏状态 → 进入主游戏循环

↓

处理鼠标输入 → 更新游戏状态 → 绘制界面（双缓冲防闪烁）

↓

30 天学习周期内：

- 点击课本：进入答题场景刷题
- 点击黑板：听课，全科目加分
- 点击手机：摸鱼休息，概率触发躲班主任小游戏
- 每 10 天触发月考
- 30% 概率触发随机事件

↓

第 30 天高考 → 进入志愿填报场景 → 选择志愿 → 生成结局 → 显示结局界面

↓

点击任意位置 → 重新初始化游戏

2. 核心模块设计

本项目采用模块化设计，共分为 6 个核心模块：

（1）游戏状态管理模块

- 定义 GameState 结构体，存储所有游戏核心数据

cpp

-

运行

-

```
struct GameState {  
  
    int day;                // 当前天数  
  
    int subjectScore[SUBJECT_COUNT]; // 各科分数  
  
    int totalScore;         // 高考总分  
  
    int fatigue;            // 疲劳值  
  
    Scene currentScene;     // 当前场景  
  
    bool gameOver;          // 游戏是否结束  
  
    int volunteerChoice;    // 志愿选择  
  
    string eventText;       // 事件提示文本  
  
    string resultText;      // 结局文本  
  
    // 答题、小游戏相关状态...};
```

-
- 提供 `InitGame()` 和 `RestartGame()` 函数，负责初始化和重置游戏状态
- **(2) 场景绘制模块**
- 核心函数: `DrawScene()`
- 功能: 根据当前游戏状态绘制不同场景
 - 教室主场景: 绘制背景、黑板、课桌、互动区域、状态栏
 - 考场场景: 绘制题目、倒计时、选项按钮
 - 志愿填报场景: 显示最终分数、志愿选项
 - 结局场景: 显示录取结果
- 实现了双缓冲绘图 (`BeginBatchDraw()/FlushBatchDraw()/EndBatchDraw()`)，彻底解决界面闪烁问题

(3) 输入处理模块

- 核心函数: `HandleMouseClicked(int x, int y)`
- 功能: 处理鼠标点击事件，根据点击位置和当前场景执行对应操作
- 支持四种场景下的输入处理: 教室场景、考场场景、志愿填报场景、结局场景

(4) 随机事件模块

- 核心函数: `RandomEvent()`
- 功能: 每天有 30% 概率触发随机事件
- 包含 11 种不同的随机事件，涵盖正面、负面和魔性梗事件
- 事件会影响玩家的分数、单科成绩和疲劳值

(5) 答题系统模块

- 核心函数: `GenerateRandomQuestion()`、`StartMonthlyExam()`、`StartFinalExam()`
- 功能:
 - 日常刷题: 15 秒限时，答对单科 +10 分
 - 月考: 20 秒限时，答对单科 +10 分
 - 高考: 30 秒限时，答对单科 +20 分
- 包含 12 道分科目题库 (语、数、英、理综各 3 题)，可自行扩展

(6) 志愿填报与结局生成模块

- 核心函数: `GenerateResult()`
- 功能: 根据玩家的最终分数和志愿选择生成对应的结局
- 包含 25 种差异化结局，覆盖从省状元到蓝翔技校的所有分数段
- 支持滑档调剂机制: 实际录取线比志愿标注线高 20 分，滑档后自动调剂到下一档

3. 核心机制与算法

(1) 疲劳值平衡机制

- 疲劳值范围: 0-100
- 不同行动对疲劳值的影响:
 - 疯狂刷题: +20 疲劳
 - 认真听课: +10 疲劳
 - 摸鱼休息: -30 疲劳
- 当疲劳值 ≥ 100 时，强制休息一天，疲劳值重置为 50
- 该机制迫使玩家在学习和休息之间做出平衡，增加了游戏的策略性

(2) 分科目成长机制

- 拆分为语文、数学、英语、理综 4 科，单科满分 150，总分 750
- 不同行动对应不同科目提升:
 - 刷题答题: 根据题目所属科目加分

- 认真听课：全科目 +5 分
- 随机事件：根据事件内容影响对应科目或总分

（3）随机事件触发机制

- 每天行动结束后，生成一个 0-9 的随机数
- 如果随机数 < 3（即 30% 概率），触发随机事件
- 随机事件 ID 从 0-10，每个事件有相等的触发概率

（4）滑档调剂机制

- 实际录取线比志愿标注的最低分数高 20 分
- 例如：清华大学标注需 680+，实际录取线为 700 分
- 如果分数不够第一志愿，自动调剂到下一档学校
- 如果所有志愿都滑档，触发 "创业成功" 特殊结局

五. 单元测试

针对各个核心模块设计了以下测试案例，所有测试均通过：

表格

测试模块	测试案例	输入操作	预期输出	实际输出	测试结果
初始化测试	游戏启动	运行程序	显示教室主界面，第 1 天，初始分 360，疲劳 0	与预期一致	通过
教室互动测试	点击课本	点击课本区域	进入考场场景，开始答题，疲劳 + 20	与预期一致	通过
	点击黑板	点击黑板区域	全科目 + 5 分，疲劳 + 10，天数 + 1	与预期一致	通过
	点击手机	点击手机区域	50% 概率触发躲班主任游戏，否则疲劳 - 30，天数 + 1	与预期一致	通过
答题系统测试	答对题目	考场场景点击正确选项	对应科目 +10/15/20 分，返回教室场景，天数 + 1	与预期一致	通过
	答错题目	考场场景点击错误选项	无加分，返回教室场景，天数 + 1	与预期一致	通过
	答题超时	考场场景等待时间结束	提示答题超时，返回教室场景，天数 + 1	与预期一致	通过
疲劳值机制测试	疲劳值满 100	连续点击 5 次课本	强制休息一天，疲劳值重置为 50	与预期一致	通过
随机事件测试	事件触发	多次点击行动按钮	约 30% 概率触发随机事件	与预期一致	通过
月考测试	第 10 天月考	游戏进行到第 10 天	自动进入月考场景，20 秒限时答题	与预期一致	通过
高考测试	第 30 天高考	游戏进行到第 30 天	自动进入高考场景，30 秒限时答题	与预期一致	通过
志愿填报测试	分数足够	总分 700，点击 "清华大学"	成功录取清华大学，显示对应结局	与预期一致	通过

测试模块	测试案例	输入操作	预期输出	实际输出	测试结果
滑档机制测试 结局生成测试	分数不够	总分 650, 点击 “清华大学”	提示分数不够, 无法选择	与预期一致	通过
	第一志愿滑档	总分 690, 点击 “清华大学”	滑档调剂到 985 大学, 显示对应结局	与预期一致	通过
	高分结局	总分 730, 选择清华大学	显示 “全省理科状元” 结局	与预期一致	通过
	低分结局	总分 200, 选择专科学校	显示 “蓝翔技工学校” 结局	与预期一致	通过
	复读结局	选择 “复读一年” 志愿	显示复读结局	与预期一致	通过
重新开始测试	游戏结束后点击	结局界面点击任意位置	游戏重置, 回到第 1 天	与预期一致	通过

六。收获与体会

通过本次大作业，我在以下几个方面获得了显著的提升：

1. EasyX 图形库的使用

- 掌握了 EasyX 图形库的基本绘图函数，包括绘制矩形、文字、填充颜色等
- 学会了如何处理鼠标输入和实现简单的 UI 交互
- 理解了图形化程序的主循环结构：输入处理→状态更新→界面绘制
- 掌握了双缓冲绘图技术（BeginBatchDraw()/FlushBatchDraw()/EndBatchDraw()），彻底解决了界面闪烁问题

2. 面向对象编程思想的实践

- 通过结构体和函数的模块化设计，体现了封装的思想
- 学会了如何将复杂的游戏逻辑拆分成多个独立的函数（如 DrawScene()、HandleMouseClicked()、RandomEvent() 等），提高了代码的可读性和可维护性
- 理解了状态管理在游戏开发中的重要性，通过 GameState 结构体统一管理所有游戏数据

3. 游戏设计与用户体验

- 学会了如何设计简单的游戏机制，让游戏既有挑战性又有趣味性
- 理解了用户体验的重要性，比如互动区域的大小和位置要适合鼠标点击，文本要清晰易读
- 掌握了如何通过随机事件、答题系统和多结局系统增加游戏的 replay value

4. 调试与测试

- 学会了如何设计单元测试案例，覆盖各个功能模块
- 掌握了调试图形化程序的方法，比如通过输出日志定位问题
- 提高了排查和解决 bug 的能力

遇到的难点及解决方法

- 界面闪烁问题：**一开始直接在屏幕上绘制，闪烁严重。后来通过学习 EasyX 文档，使用双缓冲绘图技术，先在后台绘制完整帧再一次性刷新到屏幕，彻底解决了闪烁问题。
- 场景切换逻辑：**一开始场景切换的条件判断比较混乱，后来通过定义 Scene 枚举和 currentScene 变量，统一管理当前场景，简化了逻辑。
- 随机事件和答题系统的整合：**需要确保在答题或小游戏进行时不会触发其他事件，通过 isAnswering 和 isPlayingHideGame 等状态变量进行控制，避免了逻辑冲突。

4. **滑档调剂逻辑：**一开始滑档逻辑比较复杂，后来通过先判断是否滑档，再逐级调剂的方式，简化了代码结构，提高了可读性。

本次大作业让我将课堂上学到的 C++ 知识应用到了实际项目中，不仅巩固了语法基础，还提高了工程实践能力，为后续的专业课程学习打下了坚实的基础。

七. AI 工具使用披露

- **AI 工具名称及版本：** gemini3.1pro
- **具体用途：**
 1. 代码开发：协助编写核心游戏逻辑、界面绘制
 2. 功能设计：提供游戏玩法和爆点设计的建议